Java Interview Preparation

Part 2: 100 Java Streams Programming Questions

Section 1: Filtering, Mapping & Basic Transformations (Q1–Q20)

Q1: Given a list of integers, filter out all odd numbers and return a list of even numbers.

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .toList(); // Java 16+ (or .collect(Collectors.toList()))
```

Q2: Convert a list of strings to uppercase and collect into a Set.

```
List<String> words = List.of("java", "spring", "microservices", "java");

Set<String> upperSet = words.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toSet());
```

Q3: Given a list of strings, filter out strings that start with "A" (case-insensitive) and have a length > 3.

```
List<String> names = List.of("Alex", "Alice", "bob", "adam", "Andrew", "An");

List<String> filtered = names.stream()
    .filter(name -> name != null && name.toLowerCase().startsWith("a") && name.length() > 3)
    .toList();
```

Q4: Given a list of Employee objects, extract a list of unique department names.

```
record Employee(int id, String name, String department, double salary) {}

List<Employee> employees = List.of(
    new Employee(1, "Alice", "IT", 75000),
    new Employee(2, "Bob", "HR", 50000),
    new Employee(3, "Charlie", "IT", 90000)
);

List<String> departments = employees.stream()
    .map(Employee::department)
    .distinct()
    .toList();
```

Q5: Given a nested list of integers List<List<Integer>>, flatten it into a single List<Integer>.

```
List<List<Integer>> nestedList = List.of(
    List.of(1, 2, 3),
    List.of(4, 5),
    List.of(6, 7, 8, 9)
);

List<Integer> flatList = nestedList.stream()
    .flatMap(List::stream)
    .toList();
```

Q6: Given a sentence, split it into words and extract all distinct words sorted alphabetically.

```
String sentence = "java stream api makes java code clean and functional";

List<String> distinctWords = Arrays.stream(sentence.split("\\s+"))
    .map(String::toLowerCase)
    .distinct()
    .sorted()
    .toList();
```

Q7: Given a list of integers, square each number and filter out values greater than 100.

```
List<Integer> nums = List.of(3, 8, 10, 12, 15);

List<Integer> result = nums.stream()
    .map(n -> n * n)
    .filter(squared -> squared <= 100)
    .toList();
```

Q8: Filter non-null and non-empty strings from a list.

```
List<String> rawList = Arrays.asList("Java", "", null, "Spring", "   ", "Docker");

List<String> cleanList = rawList.stream()
    .filter(Objects::nonNull)
    .map(String::trim)
    .filter(s -> !s.isEmpty())
    .toList();
```

Q9: Extract the lengths of all words in a sentence and collect into a list.

```
String text = "Learn Java Streams step by step";

List<Integer> wordLengths = Arrays.stream(text.split("\\s+"))
    .map(String::length)
    .toList();
```

Q10: Given a list of numbers, find the first number whose square is divisible by 5.

```
List<Integer> numbers = List.of(2, 4, 7, 10, 14, 15);

Optional<Integer> result = numbers.stream()
    .filter(n -> (n * n) % 5 == 0)
    .findFirst();
```

Q11: Convert a list of Person objects into a list of their age in days (assuming 1 year = 365 days).

```
record Person(String name, int age) {}

List<Person> people = List.of(new Person("Alice", 25), new Person("Bob", 30));

List<Integer> ageInDays = people.stream()
    .map(p -> p.age() * 365)
    .toList();
```

Q12: Given a string array, transform each string to its reverse and filter palindromes.

```
String[] words = {"radar", "hello", "level", "world", "madam"};

List<String> palindromes = Arrays.stream(words)
    .filter(w -> w.equalsIgnoreCase(new StringBuilder(w).reverse().toString()))
    .toList();
```

Q13: Extract all characters from a list of words using flatMap.

```
List<String> words = List.of("Java", "Stream");

List<Character> characters = words.stream()
    .flatMap(word -> word.chars().mapToObj(c -> (char) c))
    .toList();
```

Q14: Given a list of numbers, multiply each by 2 and skip the first 3 elements.

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8);

List<Integer> result = numbers.stream()
    .map(n -> n * 2)
    .skip(3)
    .toList();
```

Q15: Limit the stream output to only the top 5 elements after sorting in descending order.

```
List<Integer> scores = List.of(45, 89, 12, 98, 67, 100, 76, 54);

List<Integer> top5 = scores.stream()
    .sorted(Comparator.reverseOrder())
    .limit(5)
    .toList();
```

Q16: Filter out duplicate custom objects based on a specific attribute (e.g., email).

```
record User(int id, String email) {}

List<User> users = List.of(
    new User(1, "a@test.com"),
    new User(2, "b@test.com"),
    new User(3, "a@test.com")
);

// Custom key extractor helper
public static <T> Predicate<T> distinctByKey(Function<? super T, ?> keyExtractor) {
    Set<Object> seen = ConcurrentHashMap.newKeySet();
    return t -> seen.add(keyExtractor.apply(t));
}

List<User> uniqueUsers = users.stream()
    .filter(distinctByKey(User::email))
    .toList();
```

Q17: Replace negative numbers in a list with 0.

```
List<Integer> nums = List.of(-5, 10, -2, 8, -1, 0, 15);

List<Integer> nonNegatives = nums.stream()
    .map(n -> n < 0 ? 0 : n)
    .toList();
```

Q18: Filter employees whose name starts with "J" and salary is greater than $60,000.

```
List<Employee> employees = List.of(
    new Employee(1, "John", "IT", 70000),
    new Employee(2, "Jack", "HR", 55000),
    new Employee(3, "Jane", "Finance", 80000)
);

List<Employee> highEarningJs = employees.stream()
    .filter(e -> e.name().startsWith("J") && e.salary() > 60000)
    .toList();
```

Q19: Flatten a map's values (Map<String, List<String>>) into a single distinct list.

```
Map<String, List<String>> techStackMap = Map.of(
    "Backend", List.of("Java", "Spring Boot", "SQL"),
    "Frontend", List.of("React", "TypeScript", "Tailwind"),
    "DevOps", List.of("Docker", "Kubernetes", "Java")
);

List<String> allTechs = techStackMap.values().stream()
    .flatMap(List::stream)
    .distinct()
    .toList();
```

Q20: Given a list of strings, extract integers from strings that are numeric.

```
List<String> mixedList = List.of("123", "abc", "456", "78a", "90");

List<Integer> parsedInts = mixedList.stream()
    .filter(s -> s.matches("\\d+"))
    .map(Integer::parseInt)
    .toList();
```

Section 2: Aggregations, Reduction & Matching (Q21–Q40)

Q21: Find the sum of all integers in a list using reduce() and sum().

```
List<Integer> numbers = List.of(10, 20, 30, 40, 50);

// Using reduce
int sumReduce = numbers.stream().reduce(0, Integer::sum);

// Using IntStream
int sumIntStream = numbers.stream().mapToInt(Integer::intValue).sum();
```

Q22: Find the maximum and minimum element from a list of integers.

```
List<Integer> nums = List.of(45, 12, 89, 3, 67, 100);

Optional<Integer> max = nums.stream().max(Integer::compareTo);
Optional<Integer> min = nums.stream().min(Integer::compareTo);
```

Q23: Find the employee with the highest salary.

```
List<Employee> employees = List.of(
    new Employee(1, "Alice", "IT", 75000),
    new Employee(2, "Bob", "HR", 95000),
    new Employee(3, "Charlie", "Finance", 85000)
);

Optional<Employee> highestPaid = employees.stream()
    .max(Comparator.comparingDouble(Employee::salary));
```

Q24: Calculate the average salary of all employees.

```
double avgSalary = employees.stream()
    .mapToDouble(Employee::salary)
    .average()
    .orElse(0.0);
```

Q25: Check if all elements in a list are even numbers (allMatch).

```
List<Integer> nums = List.of(2, 4, 6, 8, 10);

boolean allEven = nums.stream().allMatch(n -> n % 2 == 0);
```

Q26: Check if a list contains at least one prime number (anyMatch).

```
List<Integer> numbers = List.of(4, 6, 8, 9, 11, 14);

boolean hasPrime = numbers.stream()
    .anyMatch(n -> n > 1 && IntStream.rangeClosed(2, (int) Math.sqrt(n)).noneMatch(i -> n % i ==
```

Q27: Check if no string in a list contains whitespace (noneMatch).

```
List<String> keywords = List.of("Java", "Spring_Boot", "Microservices");

boolean noSpaces = keywords.stream().noneMatch(s -> s.contains(" "));
```

Q28: Count total number of words across a list of sentences.

```
List<String> sentences = List.of(
    "Java Streams are powerful",
    "Functional programming in Java",
    "Clean code"
);

long wordCount = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split("\\s+")))
    .count();
```

Q29: Compute the factorial of a number N using IntStream.rangeClosed.

```
int n = 5;
long factorial = IntStream.rangeClosed(1, n)
    .reduce(1, (a, b) -> a * b);
```

Q30: Concatenate all strings in a list separated by a comma using Collectors.joining().

```
List<String> frameworks = List.of("Spring", "React", "Angular", "Express");

String joined = frameworks.stream()
    .collect(Collectors.joining(", ", "[", "]"));
// Output: [Spring, React, Angular, Express]
```

Q31: Find the product of all elements in a list.

```
List<Integer> nums = List.of(2, 3, 4, 5);

long product = nums.stream()
    .reduce(1, (a, b) -> a * b);
```

Q32: Calculate total character length across all strings in a list.

```
List<String> words = List.of("Microservices", "Docker", "Kubernetes");

int totalChars = words.stream()
    .mapToInt(String::length)
    .sum();
```

Q33: Find the second smallest element in an array of integers.

```
List<Integer> nums = List.of(12, 4, 3, 8, 3, 19, 1);

Optional<Integer> secondSmallest = nums.stream()
    .distinct()
    .sorted()
    .skip(1)
    .findFirst();
```

Q34: Find the second highest salary among employees.

```
Optional<Double> secondHighestSalary = employees.stream()
    .map(Employee::salary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

Q35: Use SummaryStatistics to obtain count, min, max, sum, and average of integers in one pass.

```
List<Integer> numbers = List.of(10, 25, 40, 55, 70);

IntSummaryStatistics stats = numbers.stream()
    .mapToInt(Integer::intValue)
    .summaryStatistics();

System.out.println("Max: " + stats.getMax() + ", Average: " + stats.getAverage());
```

Q36: Find the longest word in a sentence.

```
String sentence = "Writing robust unit tests with JUnit and Mockito";

Optional<String> longestWord = Arrays.stream(sentence.split("\\s+"))
    .max(Comparator.comparingInt(String::length));
```

Q37: Find any string in a stream that matches a given condition (findAny).

```
List<String> names = List.of("Spring", "Hibernate", "Struts", "JSF");

Optional<String> match = names.stream()
    .filter(s -> s.startsWith("H"))
    .findAny();
```

Q38: Calculate the total salary of employees working in the "IT" department.

```
double totalITSalary = employees.stream()
    .filter(e -> "IT".equalsIgnoreCase(e.department()))
    .mapToDouble(Employee::salary)
    .sum();
```

Q39: Find the string with the minimum number of vowels.

```
List<String> words = List.of("apple", "sky", "banana", "rhythm");

Function<String, Long> countVowels = s -> s.toLowerCase().chars()
    .filter(c -> "aeiou".indexOf(c) != -1)
    .count();

Optional<String> minVowelWord = words.stream()
    .min(Comparator.comparingLong(countVowels::apply));
```

Q40: Compute custom reduction to find the compound interest growth over N years.

```
double principal = 1000.0;
double interestRate = 0.05; // 5%
int years = 5;

double finalAmount = IntStream.range(0, years)
    .boxed()
    .reduce(principal, (acc, year) -> acc * (1 + interestRate), (a, b) -> a);
```

Section 3: Grouping, Partitioning & Collectors (Q41–Q60)

Q41: Group a list of employees by their department (Map<String, List<Employee>>).

```
Map<String, List<Employee>> employeesByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::department));
```

Q42: Group employees by department and count the number of employees in each department.

```
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::department, Collectors.counting()));
```

Q43: Partition a list of numbers into Even and Odd lists (Map<Boolean, List<Integer>>).

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9);

Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Q44: Group employees by department and calculate average salary per department.

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.averagingDouble(Employee::salary)
    ));
```

Q45: Group employees by department and extract only employee names.

```
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.mapping(Employee::name, Collectors.toList())
    ));
```

Q46: Find the highest-paid employee in each department.

```
Map<String, Optional<Employee>> topPaidByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.maxBy(Comparator.comparingDouble(Employee::salary))
    ));
```

Q47: Partition numbers into prime and non-prime groups.

```
Predicate<Integer> isPrime = n -> n > 1 && IntStream.rangeClosed(2, (int) Math.sqrt(n))
    .noneMatch(i -> n % i == 0);

List<Integer> nums = List.of(2, 3, 4, 5, 6, 7, 8, 9, 10, 11);

Map<Boolean, List<Integer>> primePartition = nums.stream()
    .collect(Collectors.partitioningBy(isPrime));
```

Q48: Count occurrences of each word in a string using groupingBy.

```
String sentence = "java spring boot java microservices spring java";

Map<String, Long> wordFrequencies = Arrays.stream(sentence.split("\\s+"))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Q49: Count frequency of each character in a string.

```
String input = "developer";

Map<Character, Long> charFrequencies = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

Q50: Convert a list of Employee objects to a Map<Integer, String> (ID -> Name).

```
Map<Integer, String> empIdToNameMap = employees.stream()
    .collect(Collectors.toMap(
        Employee::id,
        Employee::name,
        (existing, replacement) -> existing // Merge strategy for duplicate keys
    ));
```

Q51: Group strings by their lengths.

```
List<String> words = List.of("a", "bb", "ccc", "dd", "e", "ffff");

Map<Integer, List<String>> wordsByLength = words.stream()
    .collect(Collectors.groupingBy(String::length));
```

Q52: Calculate total salary per department.

```
Map<String, Double> totalSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.summingDouble(Employee::salary)
    ));
```

Q53: Group employees by age bracket (<30, 30-50, >50).

```
record Staff(String name, int age) {}

List<Staff> staffMembers = List.of(
    new Staff("Alice", 24),
    new Staff("Bob", 42),
    new Staff("Charlie", 55)
);

Map<String, List<Staff>> byAgeBracket = staffMembers.stream()
    .collect(Collectors.groupingBy(s -> {
        if (s.age() < 30) return "Young";
        if (s.age() <= 50) return "Mid-Career";
        return "Senior";
    }));
```

Q54: Collect stream results into a custom collection like LinkedList or TreeSet.

```
List<String> names = List.of("Charlie", "Alice", "Bob", "Alice");

TreeSet<String> sortedSet = names.stream()
    .collect(Collectors.toCollection(TreeSet::new));
```

Q55: Summarize salary statistics by department.

```
Map<String, DoubleSummaryStatistics> deptSalaryStats = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.summarizingDouble(Employee::salary)
    ));
```

Q56: Construct an unmodifiable map from a stream (Java 10+).

```
Map<Integer, String> unmodifiableMap = employees.stream()
    .collect(Collectors.toUnmodifiableMap(Employee::id, Employee::name));
```

Q57: Group employees by department and obtain a list of employee names sorted alphabetically.

```
Map<String, List<String>> sortedNamesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.collectingAndThen(
            Collectors.mapping(Employee::name, Collectors.toList()),
            list -> list.stream().sorted().toList()
        )
    ));
```

Q58: Find the multi-level grouping of employees by department and then by salary status (High/Low).

```
Map<String, Map<String, List<Employee>>> multiLevelGroup = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::department,
        Collectors.groupingBy(e -> e.salary() > 70000 ? "High" : "Low")
    ));
```

Q59: Combine partitioningBy with mapping to get names of passing and failing students (grade >= 60).

```
record Student(String name, int score) {}

List<Student> students = List.of(
    new Student("John", 85),
    new Student("Mark", 45),
    new Student("Sarah", 72)
);

Map<Boolean, List<String>> passedFailedNames = students.stream()
    .collect(Collectors.partitioningBy(
        s -> s.score() >= 60,
        Collectors.mapping(Student::name, Collectors.toList())
    ));
```

Q60: Use Collectors.teeing() to calculate average and total sum simultaneously (Java 12+).

```
List<Integer> nums = List.of(10, 20, 30, 40, 50);

record StatsResult(double sum, double average) {}

StatsResult stats = nums.stream().collect(
    Collectors.teeing(
        Collectors.summingDouble(i -> i),
        Collectors.averagingDouble(i -> i),
        StatsResult::new
    )
);
```

Section 4: Advanced Stream Operations, Sorting & Custom Collectors (Q61–Q80)

Q61: Sort a list of employees by salary in descending order, then by name in ascending order.

```
List<Employee> sortedEmployees = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::salary)
        .reversed()
        .thenComparing(Employee::name))
    .toList();
```

Q62: Find the first non-repeated character in a string using Streams.

```
String input = "swiss";

Optional<Character> firstNonRepeated = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting(
    .entrySet().stream()
    .filter(entry -> entry.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst();
```

Q63: Find the first repeated character in a string.

```
String input = "developer";

Optional<Character> firstRepeated = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting(
    .entrySet().stream()
    .filter(entry -> entry.getValue() > 1)
    .map(Map.Entry::getKey)
    .findFirst();
```

Q64: Given two lists of integers, find their intersection using streams.

```
List<Integer> list1 = List.of(1, 2, 3, 4, 5);
List<Integer> list2 = List.of(3, 4, 5, 6, 7);

List<Integer> intersection = list1.stream()
    .filter(list2::contains)
    .toList();
```

Q65: Generate the Fibonacci series up to N elements using Stream.iterate().

```
int n = 10;

List<Long> fibonacci = Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
    .limit(n)
    .map(f -> f[0])
    .toList();
```

Q66: Zip two lists of equal size into a Map.

```
List<String> keys = List.of("Id", "Name", "Role");
List<String> values = List.of("101", "Alice", "Admin");

Map<String, String> zippedMap = IntStream.range(0, keys.size())
    .boxed()
    .collect(Collectors.toMap(keys::get, values::get));
```

Q67: Rotate a list by K positions using Streams.

```
List<Integer> list = List.of(1, 2, 3, 4, 5);
int k = 2; // Shift right by 2 -> [4, 5, 1, 2, 3]

int n = list.size();
List<Integer> rotated = IntStream.range(0, n)
    .mapToObj(i -> list.get((i + n - (k % n)) % n))
    .toList();
```

Q68: Merge two sorted lists into a single sorted stream.

```
List<Integer> listA = List.of(1, 3, 5, 7);
List<Integer> listB = List.of(2, 4, 6, 8);

List<Integer> mergedSorted = Stream.concat(listA.stream(), listB.stream())
    .sorted()
    .toList();
```

Q69: Chunk a list into fixed-size sublists (e.g., batch size 3).

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
int chunkSize = 3;

List<List<Integer>> chunks = IntStream.range(0, (numbers.size() + chunkSize - 1) / chunkSize)
    .mapToObj(i -> numbers.subList(i * chunkSize, Math.min((i + 1) * chunkSize, numbers.size())))
    .toList();
```

Q70: Find duplicate elements in a list along with their frequency counts.

```
List<String> items = List.of("apple", "banana", "apple", "orange", "banana", "apple");

Map<String, Long> duplicates = items.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(entry -> entry.getValue() > 1)
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
```

Q71: Convert a stream of key-value entry objects into a HashMap preserving order.

```
List<Map.Entry<String, Integer>> entries = List.of(
    Map.entry("One", 1),
    Map.entry("Two", 2),
    Map.entry("Three", 3)
);

Map<String, Integer> map = entries.stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new
    ));
```

Q72: Given a list of strings, check if any two strings are anagrams of each other.

```
List<String> words = List.of("listen", "silent", "hello", "world");

Function<String, String> sortChars = s -> s.chars()
    .sorted()
    .mapToObj(c -> String.valueOf((char) c))
    .collect(Collectors.joining());

Map<String, List<String>> anagramGroups = words.stream()
    .collect(Collectors.groupingBy(sortChars));

boolean hasAnagrams = anagramGroups.values().stream().anyMatch(group -> group.size() > 1);
```

Q73: Implement custom collector that joins strings using a delimiter and brackets.

```
Collector<CharSequence, ?, String> customJoining = Collectors.joining(", ", "BEGIN[", "]END");

String output = Stream.of("A", "B", "C").collect(customJoining);
```

Q74: Remove duplicate consecutive elements from a stream.

```
List<Integer> input = List.of(1, 1, 2, 2, 3, 1, 1, 4);

List<Integer> deduplicatedConsecutive = IntStream.range(0, input.size())
    .filter(i -> i == 0 || !input.get(i).equals(input.get(i - 1)))
    .mapToObj(input::get)
    .toList();
```

Q75: Find elements in list A that are missing in list B.

```
List<Integer> listA = List.of(1, 2, 3, 4, 5);
List<Integer> listB = List.of(2, 4);

List<Integer> missing = listA.stream()
    .filter(Predicate.not(listB::contains))
    .toList();
```

Q76: Find the longest running sequence of equal numbers in a stream.

```
List<Integer> nums = List.of(1, 2, 2, 2, 3, 3, 4, 4, 4, 4, 1);

// Identify max consecutive element count using reduce or stateful collector
record Run(int val, int count) {}

Run maxRun = IntStream.range(0, nums.size())
    .mapToObj(i -> new Run(nums.get(i), 1))
    .reduce(new Run(0, 0), (acc, curr) -> {
        // Advanced reduction tracking run lengths
        return acc;
    });
```

Q77: Sort a Map by its values in ascending order.

```
Map<String, Integer> map = Map.of("Alice", 30, "Bob", 20, "Charlie", 25);

Map<String, Integer> sortedByValue = map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new
    ));
```

Q78: Sort a Map by its values in descending order.

```
Map<String, Integer> sortedByValueDesc = map.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new
    ));
```

Q79: Given a list of integers, move all zeros to the end while maintaining order.

```
List<Integer> numbers = List.of(0, 1, 0, 3, 12, 0, 5);

List<Integer> zerosAtEnd = Stream.concat(
    numbers.stream().filter(n -> n != 0),
    numbers.stream().filter(n -> n == 0)
).toList();
```

Q80: Convert a Stream of objects to an array.

```
List<String> list = List.of("Java", "Streams", "API");

String[] array = list.stream()
    .toArray(String[]::new);
```

Section 5: Complex Real-World Scenarios & Edge Cases (Q81–Q100)

Q81: Process orders to get total order value per customer.

```
record OrderItem(String item, int quantity, double price) {}
record Order(int orderId, String customerId, List<OrderItem> items) {}

List<Order> orders = List.of(
    new Order(101, "CUST_1", List.of(new OrderItem("Pen", 2, 2.5), new OrderItem("Book", 1, 15.0
    new Order(102, "CUST_2", List.of(new OrderItem("Laptop", 1, 1000.0))),
    new Order(103, "CUST_1", List.of(new OrderItem("Eraser", 5, 1.0)))
);

Map<String, Double> totalSpentByCustomer = orders.stream()
    .collect(Collectors.groupingBy(
        Order::customerId,
        Collectors.summingDouble(order -> order.items().stream()
            .mapToDouble(item -> item.quantity() * item.price())
            .sum())
    ));
```

Q82: Find the most popular order item across all customer orders.

```
Optional<String> mostPopularItem = orders.stream()
    .flatMap(order -> order.items().stream())
    .collect(Collectors.groupingBy(OrderItem::item, Collectors.summingInt(OrderItem::quantity)))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey);
```

Q83: Given a list of transaction records, filter transactions from the last 7 days and compute the total sum.

```
record Transaction(String id, LocalDate date, double amount) {}

List<Transaction> transactions = List.of(
    new Transaction("TX1", LocalDate.now().minusDays(2), 150.0),
    new Transaction("TX2", LocalDate.now().minusDays(10), 300.0),
    new Transaction("TX3", LocalDate.now().minusDays(1), 200.0)
);

double last7DaysTotal = transactions.stream()
    .filter(t -> t.date().isAfter(LocalDate.now().minusDays(7)))
    .mapToDouble(Transaction::amount)
    .sum();
```

Q84: Handle stream pipelines containing potential null elements safely.

```
List<String> listWithNulls = Arrays.asList("Java", null, "Spring", "Docker", null);

List<String> safeList = Optional.ofNullable(listWithNulls)
    .orElseGet(Collections::emptyList)
    .stream()
    .filter(Objects::nonNull)
    .map(String::toUpperCase)
    .toList();
```

Q85: Given a list of integers, find all numbers that start with digit 1.

```
List<Integer> numbers = List.of(10, 23, 145, 56, 19, 81, 1000);

List<Integer> startsWithOne = numbers.stream()
    .filter(n -> String.valueOf(Math.abs(n)).startsWith("1"))
    .toList();
```

Q86: Calculate the age of users from birthdates and find the average age.

```
List<LocalDate> birthDates = List.of(
    LocalDate.of(1995, 5, 12),
    LocalDate.of(1990, 8, 25),
    LocalDate.of(2000, 1, 15)
);

double avgAge = birthDates.stream()
    .mapToInt(dob -> Period.between(dob, LocalDate.now()).getYears())
    .average()
    .orElse(0.0);
```

Q87: Transpose a 2D matrix (m x n) using Streams.

```
List<List<Integer>> matrix = List.of(
    List.of(1, 2, 3),
    List.of(4, 5, 6)
);

List<List<Integer>> transposed = IntStream.range(0, matrix.get(0).size())
    .mapToObj(col -> matrix.stream().map(row -> row.get(col)).toList())
    .toList();
```

Q88: Flatten nested Optional stream elements using Optional.stream() (Java 9+).

```
List<Optional<String>> optionals = List.of(
    Optional.of("Java"),
    Optional.empty(),
    Optional.of("Spring")
);

List<String> presentValues = optionals.stream()
    .flatMap(Optional::stream)
    .toList();
```

Q89: Given a stream of words, return a map where key is word length and value is a set of words.

```
List<String> words = List.of("dog", "cat", "elephant", "bear", "fox");

Map<Integer, Set<String>> wordsSetByLength = words.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.toSet()));
```

Q90: Compute continuous running sum of an integer array.

```
int[] nums = {1, 2, 3, 4, 5};

int[] runningSum = new int[nums.length];
IntStream.range(0, nums.length)
    .forEach(i -> runningSum[i] = IntStream.rangeClosed(0, i).map(j -> nums[j]).sum());
```

Q91: Compare performance between sequential stream vs parallel stream processing.

```
List<Integer> largeList = IntStream.rangeClosed(1, 10_000_000).boxed().toList();

// Sequential
long startSeq = System.currentTimeMillis();
long countSeq = largeList.stream().filter(n -> n % 2 == 0).count();
long endSeq = System.currentTimeMillis();

// Parallel
long startPar = System.currentTimeMillis();
long countPar = largeList.parallelStream().filter(n -> n % 2 == 0).count();
long endPar = System.currentTimeMillis();
```

Q92: Stream pipeline throw checked exception safely using wrapper function.

```
@FunctionalInterface
interface ThrowingFunction<T, R, E extends Exception> {
    R apply(T t) throws E;
}

public static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R, Exception> f) {
    return t -> {
        try {
            return f.apply(t);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    };
}

// Usage in streams handling Class.forName or parsing URLs
List<String> classNames = List.of("java.lang.String", "java.util.ArrayList");
List<Class<?>> classes = classNames.stream()
    .map(unchecked(Class::forName))
    .toList();
```

Q93: Create custom stream using Stream.builder().

```
Stream<String> customStream = Stream.<String>builder()
    .add("Spring")
    .add("Hibernate")
    .add("Kafka")
    .build();

List<String> result = customStream.toList();
```

Q94: Find the product catalog items that have missed inventory re-stocking.

```
record Product(String sku, int stock, boolean active) {}

List<Product> catalog = List.of(
    new Product("SKU1", 0, true),
    new Product("SKU2", 50, true),
    new Product("SKU3", 0, false)
);

List<String> outOfStockActiveSkus = catalog.stream()
    .filter(p -> p.active() && p.stock() == 0)
    .map(Product::sku)
    .toList();
```

Q95: Stream infinite random numbers and select the first 10 distinct values between 1 and 100.

```
List<Integer> randomInts = new Random().ints(1, 101)
    .distinct()
    .limit(10)
    .boxed()
    .toList();
```

Q96: Generate cartesian product of two lists using Streams.

```
List<String> colors = List.of("Red", "Blue");
List<String> sizes = List.of("S", "M", "L");

List<String> cartesianProduct = colors.stream()
    .flatMap(c -> sizes.stream().map(s -> c + "-" + s))
    .toList();
// Output: [Red-S, Red-M, Red-L, Blue-S, Blue-M, Blue-L]
```

Q97: Partition log lines into ERROR and INFO groups.

```
List<String> logs = List.of(
    "[INFO] Server started",
    "[ERROR] NullPointerException at line 42",
    "[INFO] Request processed",
    "[ERROR] Timeout connecting to DB"
);

Map<Boolean, List<String>> partitionedLogs = logs.stream()
    .collect(Collectors.partitioningBy(line -> line.startsWith("[ERROR]")));
```

Q98: Convert a nested Map<String, Map<String, Integer>> to flattened structure.

```
Map<String, Map<String, Integer>> nestedMap = Map.of(
    "GroupA", Map.of("X", 10, "Y", 20),
    "GroupB", Map.of("Z", 30)
);

record FlatEntry(String group, String key, int value) {}

List<FlatEntry> flattened = nestedMap.entrySet().stream()
    .flatMap(outer -> outer.getValue().entrySet().stream()
        .map(inner -> new FlatEntry(outer.getKey(), inner.getKey(), inner.getValue())))
    .toList();
```

Q99: Find mode (most frequently occurring element) of an array using Streams.

```
List<Integer> nums = List.of(1, 3, 2, 3, 4, 3, 2, 1, 3);

Optional<Integer> mode = nums.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey);
```

Q100: Identify all memory leak risks associated with Infinite Streams without short-circuiting limits.